
ZADANIE REGRESJI DANYCH METEOROLOGICZNYCH Z WYKORZYSTANIEM REKURENCYJNYCH SIECI NEURONOWYCH

Autorzy

MATEUSZ GWIZDAK 252945

MACIEJ KRYSTYNIAK 252882

Prowadzący

dr inż. Andrzej Rusiecki

Kurs

Głębokie sieci neuronowe

Spis treści

1	Propozycja projektu	2
1.1	Wstępny opis projektu	2
1.2	Zbiór danych	2
1.3	Harmonogram pracy	2
2	Rekurencyjne sieci neuronowe	4
2.1	Działanie i budowa	5
2.2	LSTM	6
2.3	GRU	8
3	Implementacja	10
3.1	Informacje wstępne	10
3.2	Załączone biblioteki	10
3.3	Przygotowanie danych	11
3.4	Przygotowanie danych	12
3.5	Model sieci	13
3.6	Parametry przekazywane dla modelu sieci	14
4	Badania	16
4.1	Cel badań	16
4.2	Przebieg badań	16
4.3	Wyniki	17
4.3.1	Modele E10 U32 S12 IN168	17
4.3.2	Modele E100 U32 S12 IN168	18
4.3.3	Modele E100 U32 S24 IN168	20
4.3.4	Fragment 2	21
4.3.5	Modele E100 U128 S12 IN168	22
4.3.6	Modele E100 U128 S12 IN124	23
5	Wnioski	25

1 Propozycja projektu

1.1 Wstępny opis projektu

Celem projektu jest zaprogramowanie modelu rekurencyjnej sieci neuronowej w oparciu o bibliotekę tensorflow. Jej zadaniem będzie jak najbardziej dokładna predykcja danych meteorologicznych takich jak temperatura czy ciśnienie powietrza. Osiągnięte to zostanie poprzez zbadanie i znalezienie jak najlepszych hiperparametrów dla zastosowanej architektury. W projekcie zawarty zostanie przegląd skuteczności w zależności od doboru konfiguracji, jak i bezpośrednie porównanie dwóch najbardziej popularnych architektur rekurencyjnych sieci neuronowych, czyli GRU i LSTM, pod kątem ich skuteczności i dokładności w regresji tych danych przy najlepiej dobranych dla siebie konfiguracjach. Sieci rekurencyjne idealnie nadają się do nauki i przetwarzania danych sekwencyjnych, takich jak przebiegi czasowe, którym jest zadany zbiór danych. Dodatkową motywacją dla zastosowania tych sieci jest to, że model pogody jest modelem nieliniowym, co sprawia duże trudności tradycyjnym sieciom i algorytmom uczenia maszynowego.

1.2 Zbiór danych

Wykorzystanym w projekcie zbiorem danych będzie "Jena Climate Dataset". Jest to zbiór danych o charakterze przebiegu czasowego zawierający 14 cech związanych z pomiarami meteorologicznymi, takimi jak temperatura powietrza, ciśnienie atmosferyczne, wilgotność powietrza, kierunek wiatru itp. Dane były zbierane co 10 minut w latach od 1 stycznia 2009 r. do 31 grudnia 2016 r. Dane zostały zaczerpnięte z portalu [kaggle.com](https://www.kaggle.com)

1.3 Harmonogram pracy

1. do 26.10 - propozycja i plan projektu,
2. do 13.11 - eksploracja i przygotowanie danych pod zadanie regresji,
3. do 27.11 - wstępna implementacja architektury LSTM i GRU,
4. do 3.12 - zbadanie architektury innej niż RNN (prawdopodobnie MLP)
5. do 8.01 - badanie skuteczności RNN, eksperymenty związane z modyfikacją hiperparametrów,

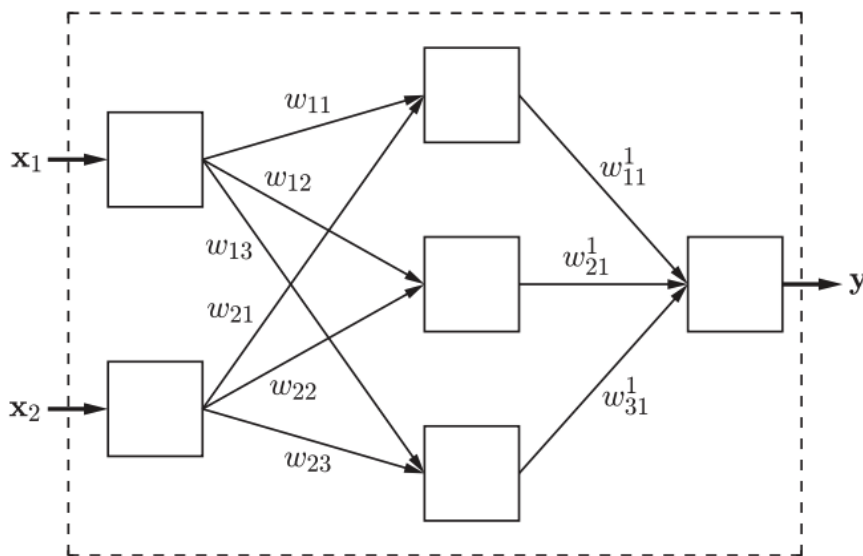
6. do 22.01 - analiza i opracowanie wyników, przygotowanie prezentacji i sprawozdania z badań.

Rezerwujemy sobie prawo do zmiany w planie projektu wraz z postępem prac.

2 Rekurencyjne sieci neuronowe

Wraz z rozwojem technologii ograniczenia, które nakładały implementacje związane z pojedynczymi perceptronami, rozpoczęły badania nad bardziej skomplikowanymi strukturami, sieciami wielowarstwowymi oraz algorytmami ich uczenia. Prosta architektura, składa się zazwyczaj z trzech warstw.

Warstwa wejściowa, odpowiadająca za wprowadzanie danych do sieci. Warstwa ukryta, odpowiadająca za realizowanie kolejnych etapów przetwarzania informacji wejściowych w informacje wyjściowe. Warstwa wyjściowa, używana do wyznaczania wartości wyjściowych sieci.



Rysunek 1: Przykładowa struktura sieci neuronowej. Posiada 2 neurony w warstwie wejściowej, 3 – w warstwie ukrytej i 1 – w warstwie wyjściowej. Jako x_1 oraz x_2 oznaczono wejścia sieci natomiast jako y wyliczone wyjście.

Rekurencyjne sieci neuronowe to rodzaj sieci neuronowych, który został zaprojektowany do pracy z danymi sekwencyjnymi. Są to zestawy danych, w których wcześniejsze wartości mają wpływ na wartości występujące później. Przykładowo mogą być to dane czasowe, gdzie wartości w danym punkcie czasowym są silnie powiązane z wartościami w poprzednich punktach czasowych.

2.1 Działanie i budowa

W Rekurencyjnej sieci neuronowej każdy neuron otrzymuje wejście nie tylko z poprzedniej warstwy, ale także od samego siebie. W ten sposób zachowuje pamięć poprzednich stanów i może uwzględniać kontekst i zależności między elementami sekwencji.

Różnica w budowie między RNN a NN pojawia się w warstwie rekurencyjnej. Neuron, który się tam znajduje ma specjalną strukturę, która pozwala mu na przetwarzanie danych sekwencyjnych. Istnieją różne warianty rekurencyjnych jednostek, takie jak RNN, LSTM (Long Short-Term Memory) i GRU (Gated Recurrent Unit), z nieznanymi różnymi wariantami budowy.

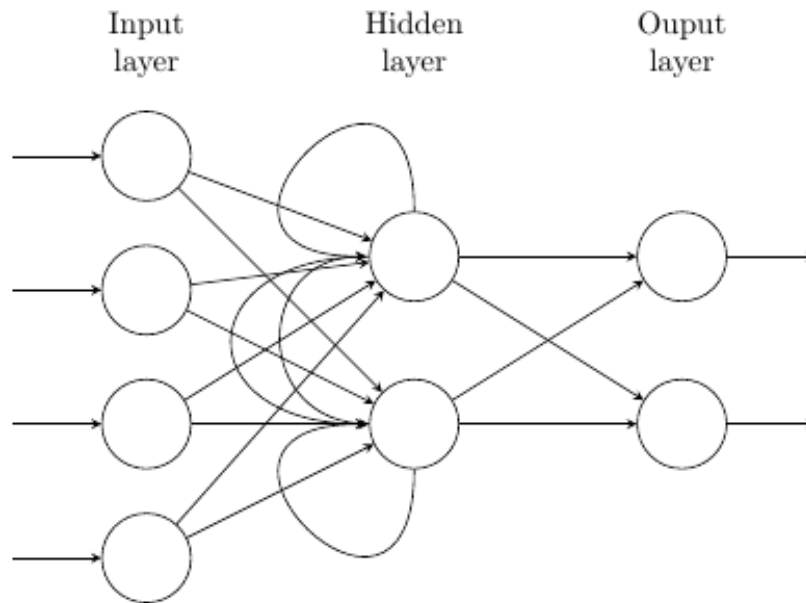


źródło: Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, O'Reilly Media, 2017.

Rysunek 2: Budowa najprostszego neuronu RNN.

Podobnie jak w podstawowej sieci neuronowej w RNN wyróżnia się trzy podstawowe warstwy: wejściową, rekurencyjną i wyjściową. Zadania warstwy wejściowej oraz wyjściowej nie zmieniają się względem podstawowego modelu sieci. Cała istota tego rozwiązania znajduje się w warstwie rekurencyjnej.

Warstwa ta składa się z neuronów rekurencyjnych. Każda rekurencyjna jednostka przyjmuje wejście z poprzedniej próbki sekwencji i aktualnego wejścia. Wyjście rekurencyjnej jednostki jest przekazywane z powrotem jako wejście dla tej samej jednostki w kolejnej próbce sekwencji.

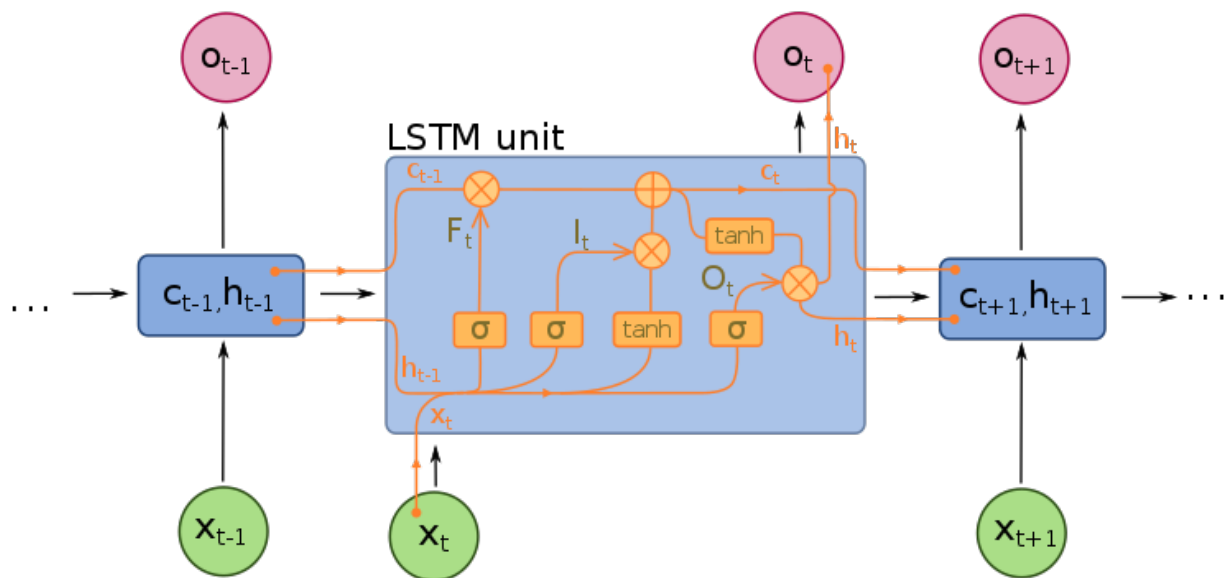


Rysunek 3: Struktura budowy rekurencyjnej sieci neuronowej.

Trzy najbardziej znane typy rekurencyjnych sieci neuronowych to prosta rekurencyjna sieć neuronowa (RNN), Long Short-Term Memory Neural Network (LSTM) i Gated Recurrent Unit (GRU).

2.2 LSTM

LSTM (Long Short-Term Memory) to specjalny rodzaj rekurencyjnej sieci neuronowej (RNN), który pozwala na przetwarzanie długich sekwencji danych z uwzględnieniem zależności czasowych między poszczególnymi elementami sekwencji.



Rysunek 4: Architektura LSTM.

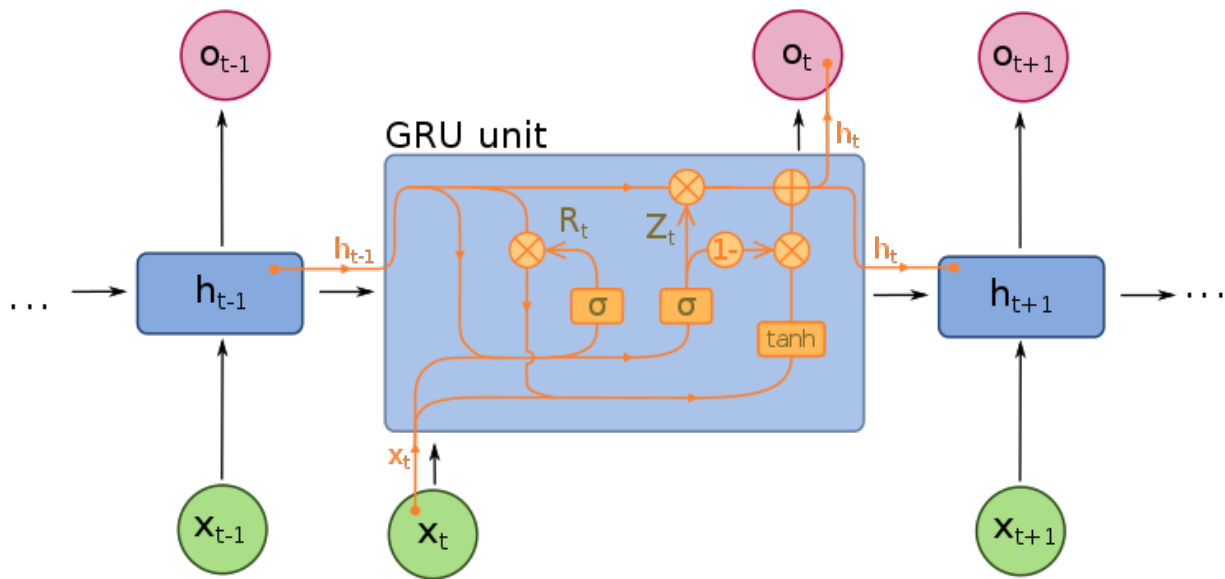
Architektura komórki LSTM jest już znacznie bardziej skomplikowana niż podstawowe jednostki RNN lub NN. Na najwyższym poziomie abstrakcji rozdziela ona wektor stanu na dwa wektory reprezentujące zależności długo $h(t)$ i krótkotrwałe $c(t)$. Podstawowa idea polega na tym, że sieć poprzez analizę stanu krótkotrwałego jest w stanie przefiltrować dane pod kątem tego, co należy zapisać w stanie długotrwałym, odrzucić, bądź co z niego odczytywać. Taki mechanizm jest możliwy dzięki zastosowaniu dodatkowych warstw służących jako kontrolery bramek.

Bramki te składają się z trzech elementów: bramki wejściowej, bramki zapominającej i bramki wyjściowej. Stan długoterminowy przechodzi przez bramkę zapomnij porzucając dane, które są już nieważne oraz dodając nowe dane, które przeszły przez bramkę wejściową. Po tej operacji wynik jest filtrowany przez bramkę wyjściową i powstaje stan krótkoterminowy.

Dzięki temu mechanizmowi, LSTM potrafi efektywnie radzić sobie z przetwarzaniem długich sekwencji danych, a także zapamiętywać informacje na dłuższy czas, co jest szczególnie przydatne w zadaniach, takich jak rozpoznawanie mowy, generowanie tekstu czy tłumaczenie maszynowe.

2.3 GRU

Analogicznie do sieci LSTM, GRU (Gated Recurrent Unit) wykorzystuje bramki, które pozwalają na kontrolowanie przepływu informacji w sieci i zapamiętywanie ważnych informacji na dłuższy czas. Bramki w GRU składają się z dwóch elementów: bramki resetowania i bramki aktualizacji. Takie podejście wprowadziło architekturę, która jest mniej złożona niż w przypadku LSTM, jednak wykazuje podobną wydajność na większości danych seryjnych, a w szczególności na danych audio i językowych.



Rysunek 5: Architektura GRU.

Bramka aktualizacji decyduje, w jakim stopniu informacje z poprzednich kroków czasowych powinny być uwzględnione w aktualnym stanie. Bramka ta analizuje poprzedni stan i bieżące wejście, a następnie generuje wartość od 0 do 1, która wskazuje, jak wiele informacji należy przekazać do aktualizacji stanu. Jeśli wartość bramki aktualizacji jest bliska 1, to znaczy, że znaczna część informacji zostanie uwzględnionych w aktualizacji stanu.

Bramka resetowania decyduje, które informacje z poprzednich kroków czasowych powinny zostać zapomniane i zresetowane. Bramka ta analizuje poprzedni stan i bieżące wejście, a następnie generuje wartość od 0 do 1, która określa, jak wiele informacji powinno zostać zapomnianych. Jeśli wartość bramki resetowania jest bliska 1, to znaczy, że większość informacji zostanie zresetowana, co pozwala na zapomnienie niepotrzebnych danych historycznych.

Nowy stan jest obliczany na podstawie aktualnego wejścia, poprzedniego stanu i informacji z aktualizacji bramki.

3 Implementacja

3.1 Informacje wstępne

Algorytm został zrealizowany przy pomocy języka programowania *Python*. Warstwy sieci neuronowych oraz funkcje, które pomogły w przygotowaniu danych do przetwarzania przez algorytm zostały zaczerpnięte z biblioteki *Tensorflow*. Dodatkowo wykorzystano biblioteki do manipulacji danymi takie jak *pandas* i *NumPy*. Pomocne okazały się również biblioteki *matplotlib* pozwalająca na graficzną reprezentację wyników predykcji oraz *pickle* umożliwiająca zapisanie obiektów generowanych w trakcie życia programu (takich jak tablica historii uczenia modelu) na dysku jako pliki.

Kluczowe dla działania algorytmu i przeprowadzania badań okazało się również zainstalowanie odpowiednich sterowników architektury *CUDA*, które wykorzystują procesor karty graficznej (w tym przypadku NVIDIA GTX 1050Ti) do wykonywania obliczeń numerycznych, co znacznie poprawiło wydajność i czas nauki generowanego modelu. Decyzja ta została podjęta po wstępnych badaniach, które zajmowały bardzo dużo czasu. Po odpowiedniej instalacji i konfiguracji czas wykonywania uczenia został skrócony nawet 10-krotnie w niektórych przypadkach.

Jeśli chodzi o dane wejściowe (treningowe) to są one podzielone na sekwencje o określonej długości (ilość wejść i ilość wyjść) i na takich danych operuje RNN. Dodatkowo sekwencje mogą być przesunięte o zadaną wartość (odległość między początkowymi punktami sekwencji). Na wyjściu otrzymujemy wektor predykcji określonej cechy dla zadanego wektora wejściowego.

Poniżej umieszczono istotne fragmenty kodu realizującego algorytm predykcji dla generowanych przebiegów dynamicznych (cały kod został załączony wraz z dokumentacją). Programy realizujące predykcję na innych zbiorach danych mogą się nieznacznie różnić. Najważniejszą różnicą jest brak skalowania danych w przypadku generowanych przebiegów, która występowała w pozostałych przypadkach.

Zaimplementowany kod oraz niektóre modele zapisane podczas eksperymentów znajdują się w repozytorium pod [linkiem](#).

3.2 Załączone biblioteki

```
1 import os
2 os.add_dll_directory("D:/CUDA/bin")
3 os.add_dll_directory("D:/CUDnn/cudnn-windows-x86_64-8.6.0.163_cuda11-archive/bin")
4 import matplotlib as mpl
5 import matplotlib.pyplot as plt
```

```
6 import numpy as np
7 import pandas as pd
8 import tensorflow as tf
9 import pickle
```

3.3 Przygotowanie danych

Jak wspomniano w propozycji projektu, do badań wykorzystano zbiór "Jena Climate Dataset". Są to dane meteorologiczne zbierane między rokiem 2009 a 2016. W ich skład wchodzi następujące zmienne:

- Date Time [dd.MM.yyyy hh:mm:ss] - znacznik momentu zebrania danych,
- p [mbar] - ciśnienie,
- T [°C] - temperatura w stopniach Celsjusza,
- Tpot [K] - temperatura w Kelwinach,
- Tdew [°C] - ciśnieniowy punkt rosy,
- rh [%] - wilgotność względna,
- VPmax [mbar] - wskaźnik maksymalnego ciśnienia pary wodnej,
- VPact - aktualne ciśnienie pary wodnej,
- VPdef - deficyt pary wodnej w powietrzu,
- sh [g/kg] - wilgotność właściwa,
- H2OC [mmol/mol] - stężenie pary wodnej,
- rho [g/m³] - gęstość powietrza,
- wv [m/s] - prędkość wiatru,
- max. wv [m/s] - maksymalna prędkość wiatru,
- wd [°] - kierunek wiatru.

Pierwszym etapem związanym z przekształceniem danych był fakt zbierania ich co 10 min. Zdecydowano się na zbieranie co 6 próbki w celu uzyskania zbioru danych z pomiarami co godzinę.

```
1 df = pd.read_csv(csv_path)
2 df = df[5::6]
```

Kolenie przygotowano dane dotyczące wiatru. Dla zmiennych `wv` i `max. wv` pojawiają się błędne wartości `-9999`, które zmieniono na wartość `0`.

```
1 #przykład usuwania anomalii dla predkosci wiatru
2 wv = df['wv (m/s)']
3 bad_wv = wv == -9999.0
4 wv[bad_wv] = 0.0
```

Dodatkowo należało przekształcić wartości kierunku wiatru. Było to spowodowane kwestiami interpretacyjnymi sieci. W rzeczywistości kierunek wyznaczany jest 1° jest bardzo zbliżony do np. 359° , co niekoniecznie poprawnie zinterpretuje model. W związku z tym przekształcono wartość kąta na wartości opowiadające jego sinusowi i cosinusowi.

```
1 #przykład przekszta cenia wiatru z wartosci kata i predkosci na predkosc X i Y
2 wv = df.pop('wv (m/s)')
3 wd_rad = df.pop('wd (deg)')*np.pi / 180
4 df['Wind_Cos'] = wv*np.cos(wd_rad)
5 df['Wind_Sin'] = wv*np.sin(wd_rad)
```

Podobne przekształcenie zastosowano w przypadku `dat`. Wiedząc, że dane zbierane zostały co godzinę, utworzono podobne kolumny dla dnia i roku w celu określenia okresowości między kolejnymi dniami czy latami.

```
1 #przykład utworzenia zmiennych Day Sin i Day Cos, dla zachowania podobiestwa
   między porami dnia
2 date_time = pd.to_datetime(df.pop('Date Time'), format='%d.%m.%Y %H:%M:%S')
3 timestamp_s = date_time.map(pd.Timestamp.timestamp)
4 day = 24*60*60
5 df['Day sin'] = np.sin(timestamp_s * (2 * np.pi / day))
6 df['Day cos'] = np.cos(timestamp_s * (2 * np.pi / day))
```

Ostatecznie zdecydowano się również usunąć kolumnę `Tpot` odpowiadającą za temperaturę w Kelwinach, gdyż jest ona bardzo mocno skorelowana z Temperaturą w stopniach Celsjusza. Usunięto również zmienną `Date Time`, ponieważ została ona zastąpiona przez odpowiednie numery próbek oraz nowe Zmienne Sin/Cos Day/Year.

3.4 Przygotowanie danych

Cały opisany w poprzednim rozdziale zbiór danych podzielono na zbiory treningowy, walidacyjny i testowy zgodnie z podziałem:

- Zbiór treningowy - pierwsze 70% zbioru,

- Zbiór walidacyjny - od 70% do 90%,
- Zbiór testowy - ostatnie 10%.

```
1 #podzial df na zbiory
2 n = len(df)
3 train_df = df[0:int(n*0.7)]
4 val_df = df[int(n*0.7):int(n*0.9)]
5 test_df = df[int(n*0.9):]
```

Dodatkowo przeprowadzono normalizację wszystkich zbiorów. W tym celu wykorzystano standaryzację Z opisaną wzorem:

$$z_i = \frac{x_i - \mu}{\sigma} \quad (1)$$

Gdzie:

- z_i to wartość ustandaryzowana,
- x_i to wartość nieustandaryzowana,
- μ to średnia wartości,
- σ to odchylenie standardowe.

```
1 #standaryzacja
2 train_mean = train_df.mean()
3 train_std = train_df.std()
4 train_df = (train_df - train_mean) / train_std
5 val_df = (val_df - train_mean) / train_std
6 test_df = (test_df - train_mean) / train_std
```

Jak widać w powyższym kodzie standaryzacja wszystkich zbiorów opiera się na wartości średniej i odchyleniu standardowym jedynie zbioru treningowego w celu zapewnienia 100% izolacji zbioru uczącego o walidacyjnego i testowego.

3.5 Model sieci

Dla uzyskania wyników jak najlepiej możliwych do porównania zdecydowano przygotować identyczne modele sekwencyjne. Poniżej przedstawiono kod tworzący model GRU:

```

1 #GRU
2 multi_lstm_model = tf.keras.Sequential([
3     tf.keras.layers.GRU(units = UNITS, return_sequences=False, activation = ACTIVATION
4         , kernel_regularizer = "l2"),
5     tf.keras.layers.Dense(OUT_STEPS*num_features, kernel_initializer=tf.initializers.
6         zeros()),
7     tf.keras.layers.Reshape([OUT_STEPS, num_features])
8 ])

```

Jak widać powyżej zbudowano model składający się pojedynczej warstwy LSTM/GRU, następnie jest warstwa gęsta przetwarzająca wyjście warstwy LSTM/GRU na wyjście o wielkości `OUT_STEPS * num_features` czyli oczekiwaną wielkość wyjścia - każda cecha * ilość predykowanych danych wyjściowych w sekwencji. Ostatnią warstwą jest `Reshape`, która przekształca wyjście z `Dense` na dwuwymiarową tablicę, gdzie jeden wymiar to predykowana sekwencja, a drugi to odpowiednia cecha.

3.6 Parametry przekazywane dla modelu sieci

Parametry, które postanowiono zbadać są następujące

Każdy parametr (i jego modyfikacja) ma mniejsze bądź większy wpływ na model i proces uczenia:

- **ACTIVATION** - funkcja aktywacji, przekształcają sumę ważoną wejść do jednostki (lub warstwy) na sygnał wyjściowy, który jest przekazywany do kolejnych warstw lub jednostek sieci neuronowej. Popularnym wyborem dla nieliniowych systemów jest funkcja tangensa hiperbolicznego (`tanh`), lub `ReLU`. Wykorzystywano funkcję `tanh`, jako, że podczas używania tej drugiej otrzymywano komunikaty o braku wsparcia bibliotek `CUDA` dla tego wyboru.
- **BATCH_SIZE** - to parametr określający liczbę próbek, które są przetwarzane jednocześnie w jednym kroku treningowym. Podczas treningu sieci neuronowych dane są przetwarzane paczkami (partiami), a nie pojedynczo. Każda paczka składa się z określonej liczby próbek. Im większa wartość, tym szybciej przebiega trening (ograniczeniem jest pamięć urządzenia),
- **OUT_STEPS** - ilość danych wyjściowych w sekwencji,
- **IN_STEPS** - ilość danych wejściowych w sekwencji,
- **MAX_EPOCHS** - maksymalna ilość przejść po całym zbiorze w procesie uczenia,

- `LEARNING_RATE_ADAM` - współczynnik uczenia dla optymalizatora *adam*, im większy, tym szybciej i gwałtowniej będą dokonywać się zmiany wektora wag,
- `UNITS` - odnosi się do liczby jednostek w warstwie. Jest to liczba ukrytych stanów w komórce, które pomagają w przechowywaniu informacji na temat sekwencji danych,
- `SEQ_STRIDE` - ilość próbek definiujący, przesunięcie między sekwencjami,

4 Badania

4.1 Cel badań

Celem badań było porównanie wydajności i możliwości predykcji dla przedstawionych danych dwóch różnych architektur rekurencyjnych sieci neuronowych przy możliwie bliskiej budowie samych modeli sieci i ich parametrów.

4.2 Przebieg badań

Obydwa modele zostały przetestowane dla takich samych hiperparametrów definiujących przebieg ich uczenia, co wpływało następnie na ich jakość predykcji. W założeniu przyjęto, że predykcja odbywać się na kolejnych dwudziestu czterech próbkach danych, co w rzeczywistości odzwierciedlało by 24-godzinną prognozę pogody. Predykowano jedną z 17 cech, które podawano na wejście, mianowicie wartość temperatury. W tabeli poniżej przedstawiono zaproponowane parametry i możliwości ich modyfikacji:

Learning rate (adam)	0.001
Funkcja aktywacji	tanh
Ilość epok	10
	100
	200
Ilość próbek na wejściu	24
	168
Ilość komórek LSTM/GRU w warstwie	32
	128
Wartość przesunięcia sekwencji podczas uczenia	12
	24
Ilość sekwencji w batchu	128

Tabela 1: Zbadane parametry

Zbadano wszystkie możliwości konfiguracji podanych parametrów zarówno dla architektury LSTM jak i dla GRU. Pozwoliło to na przeanalizowanie jaki niektóre parametry mają wpływ na konkretny model. Każdy model został zapisany na dysku wraz ze swoją historią uczenia dla funkcji straty oraz wykresem predykcji, które zostały wykonane w kolejnym kroku. Dla możliwości jak najlepszego porównania konfiguracji każdy model po zakończeniu uczenia wykonywał predykcję na dwóch takich samych dla każdej z nich fragmentach znajdujących się w zbiorze testowym, do którego sieć

nie miała dostępu podczas procesu uczenia. Fragmenty te zostały dobrane tak, aby badana sieć nie miała łatwego zadania i wyniki różniłyby się od siebie. W sprawozdaniu zamieszczono tylko niektóre wyniki wraz z opisami, które wydały się najciekawsze i można z nich wyciągnąć pewne wnioski. Do porównania samej dokładności predykcji wykorzystano 3 metryki:

- średni błąd bezwzględny (MAE)
- średni błąd kwadratowy (MSE)
- R^2 Score (R^2)

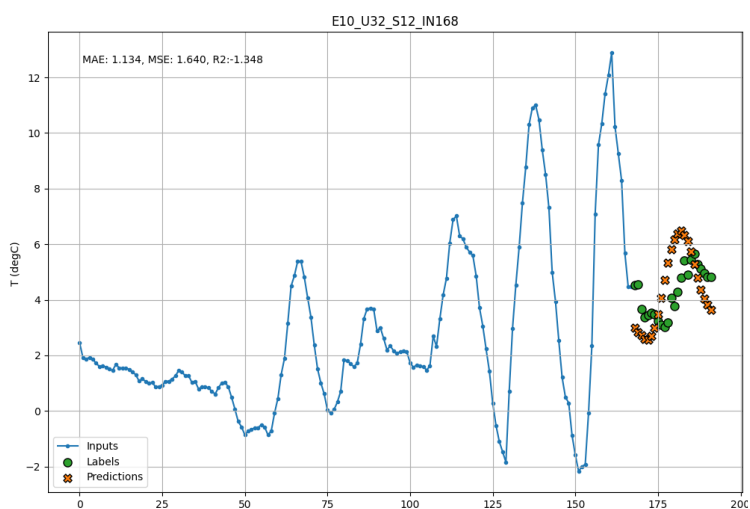
4.3 Wyniki

Dla obydwu architektur, spośród 24 modeli każdej z nich, wybrano najciekawsze wyniki, które przedstawiono poniżej. Każdemu hiperparametrowi odpowiada skrót:

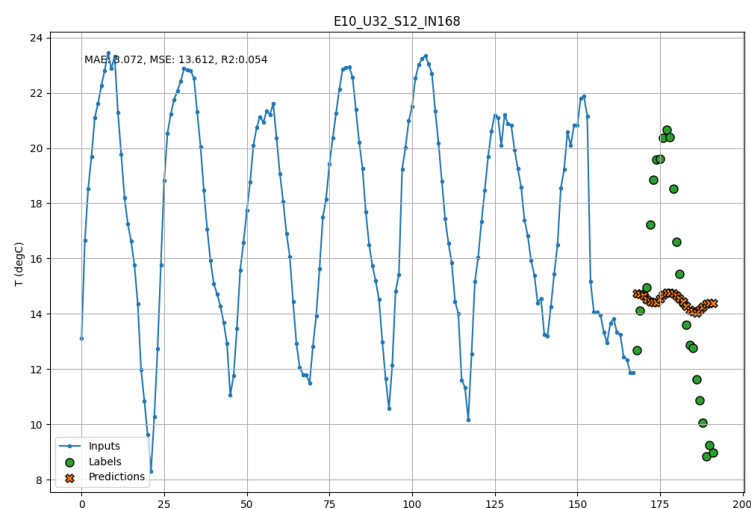
- E - ilość epok,
- U - ilość komórek LSTM/GRU w warstwie,
- S - wartość przesunięcia sekwencji w procesie uczenia,
- IN - ilość próbek wejściowych, na bazie których wykonuje się predykcja.

4.3.1 Modele E10 U32 S12 IN168

Poniżej przedstawiono wyniki działania modeli GRU oraz LSTM dla małej ilości epok.

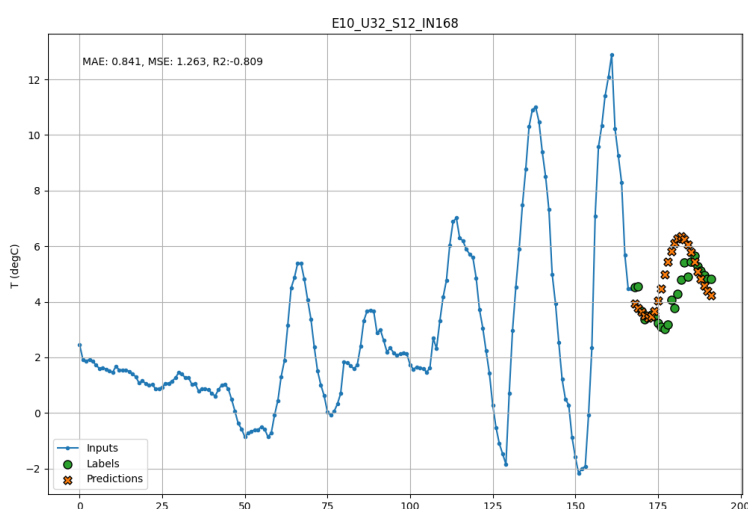


(a) Fragment 1

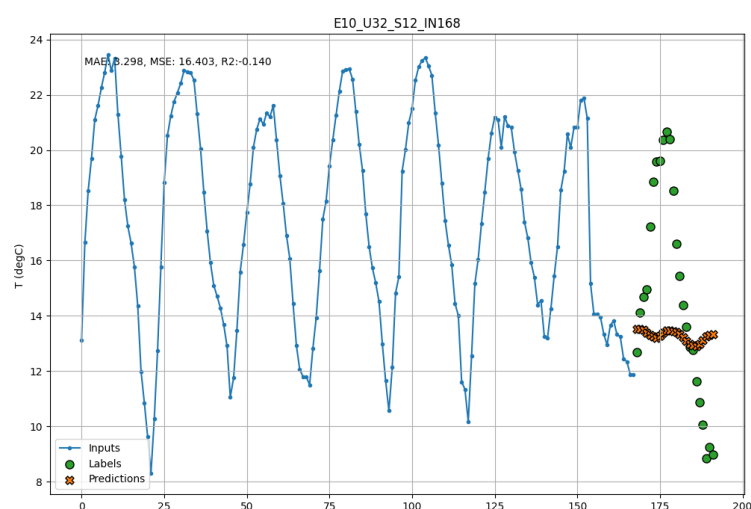


(b) Fragment 2

Rysunek 6: Predykcja GRU



(a) Fragment 1

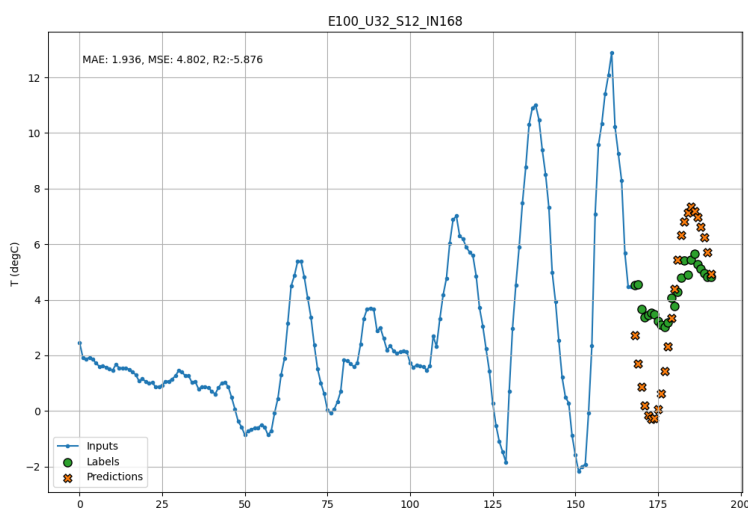


(b) Fragment 2

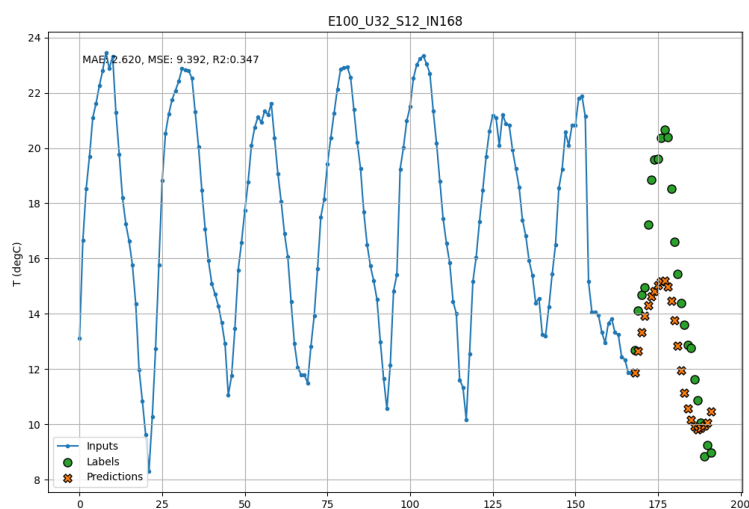
Rysunek 7: Predykcja LSTM

4.3.2 Modele E100 U32 S12 IN168

Poniższe wykresy przedstawiają te same modele lecz, uczone przez 100 epok. Jak widać znacznie poprawiono jakość predykcji obydwu modeli.

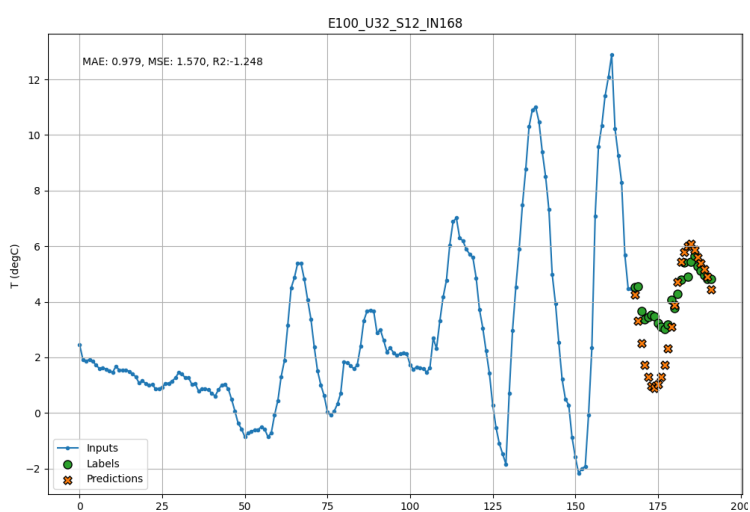


(a) Fragment 1

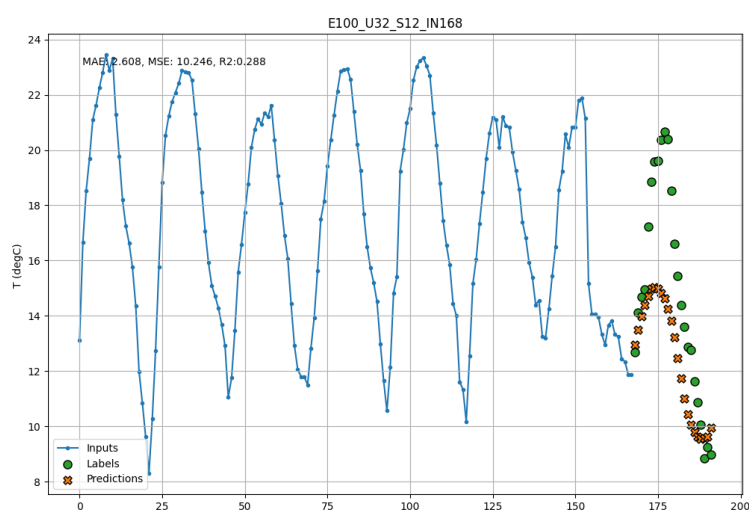


(b) Fragment 2

Rysunek 8: Predykcja GRU



(a) Fragment 1



(b) Fragment 2

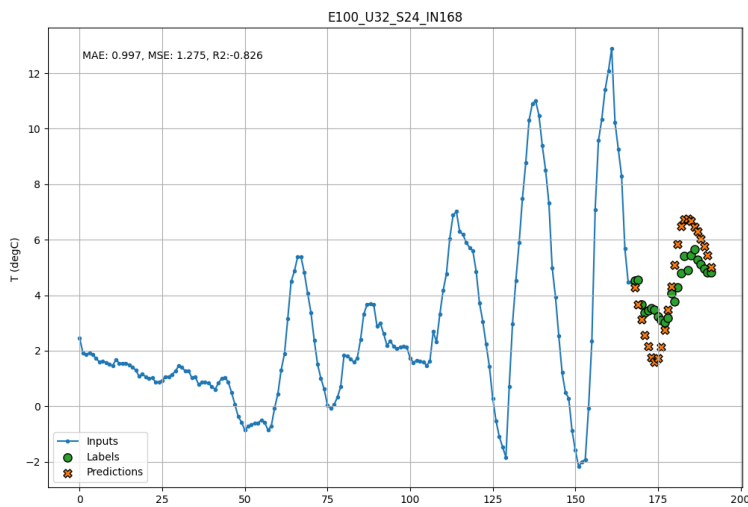
Rysunek 9: Predykcja LSTM

Jak widać na rysunku 9 obydwie modele znacznie lepiej poradziły sobie z predykcją. Ponownie bardziej dopasowany był model LSTM co również ma odzworowanie w wartościach błędu MSE:

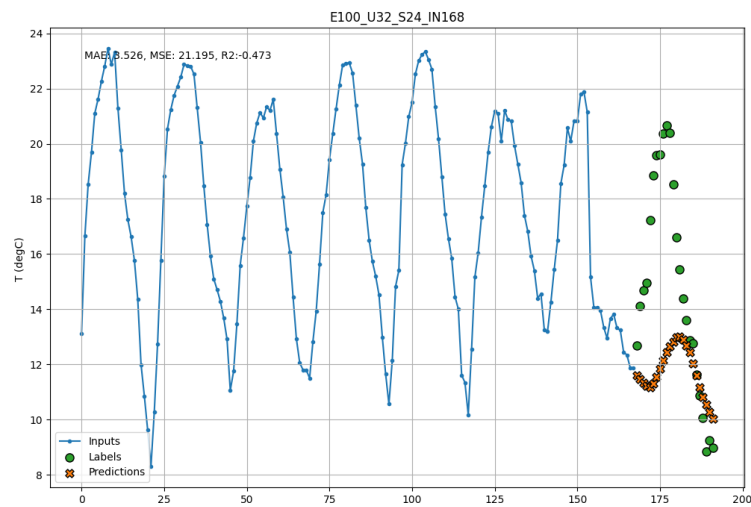
Model\Predykcja	Błąd predykcji 1	Błąd predykcji 2
GRU	4.802	9.392
LSTM	1.570	10.246

4.3.3 Modele E100 U32 S24 IN168

Kolejny model różni się od poprzedniego wartością przesunięcia sekwencji, wynosi ona 24 próbki. Teoretycznie powinno to pogorszyć jakość predykcji, ponieważ zmniejszamy ilość sekwencji uczących sieć neuronową. Jak widać na poniższych wynikach w obydwu przypadkach przypuszczenie to się sprawdziło, jednak dla sieci GRU nie miało aż tak drastycznego wpływu na jego przewidywania. Architektura LSTM ewidentnie zrobiło to gorzej, jako że jakość predykcji drastycznie spadła i wyniki są zupełnie nietrafne i odbiegające od logicznego podejścia, które zostało zachowane przez GRU.

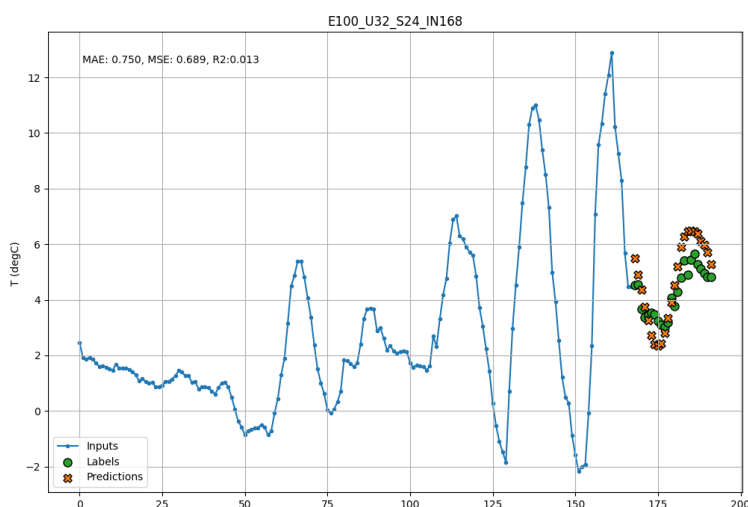


(a) Fragment 1

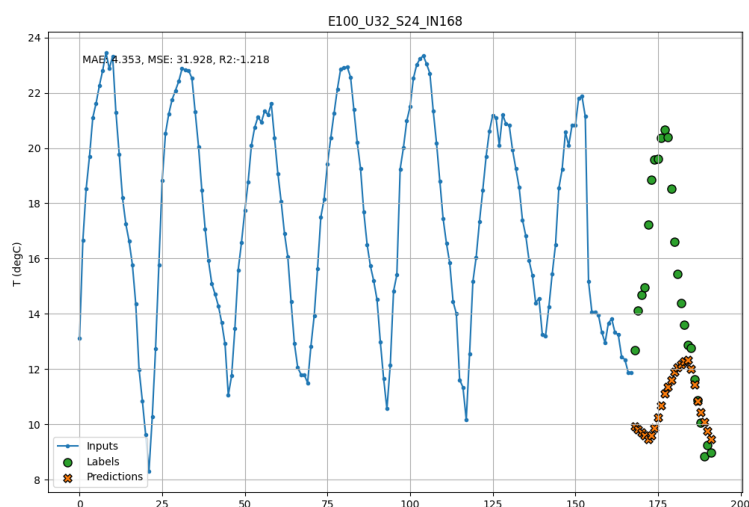


(b) Fragment 2

Rysunek 10: Predykcja GRU



(a) Fragment 1

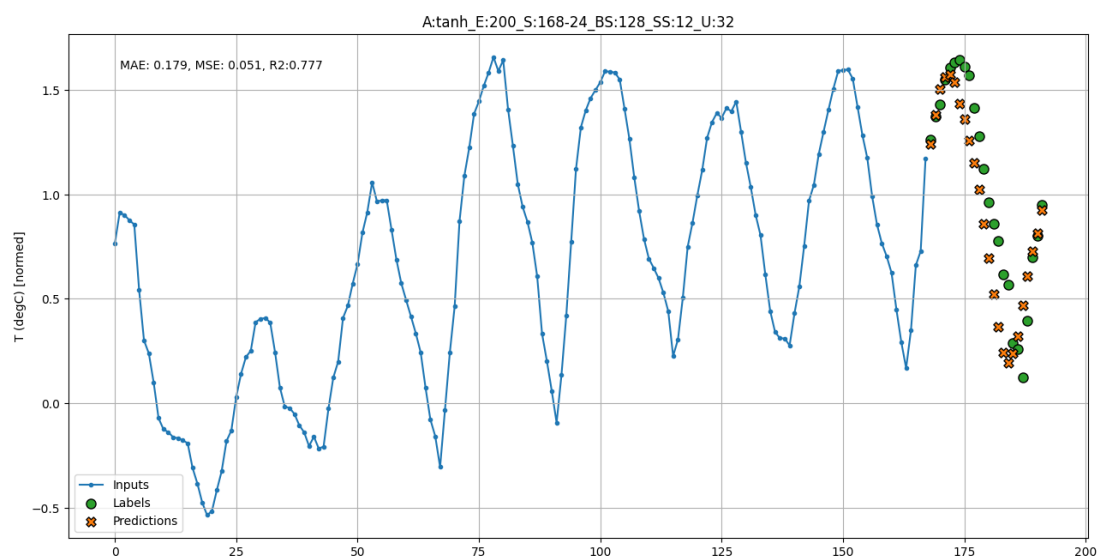


(b) Fragment 2

Rysunek 11: Predykcja LSTM

4.3.4 Fragment 2

Jak widać w poprzednich rozdziałach Fragment 2 okazał się trudniejszym do predykcji. Żaden z przedstawionych modeli nie był w stanie osiągnąć zadowalających wyników. Możliwe, że jest to spowodowane dużym nagromadzeniem punktów zaraz przed momentem predykcji - przez 13 godzin temperatura zmieniała się na przestrzeni 2 stopnie, gdzie w poprzednich dniach płynnie i ostro zmieniała się. Zdecydowano zatem przeprowadzanie testu predykcji cofając się o 3 dni - moment, w którym widoczne są całkiem płynne przejścia.

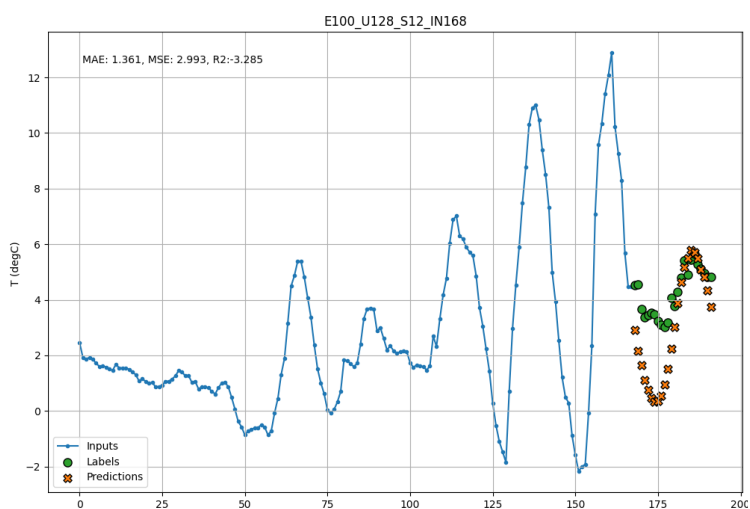


Rysunek 12: Predykcja LSTM - Fragment 2 - 3 dni wcześniej

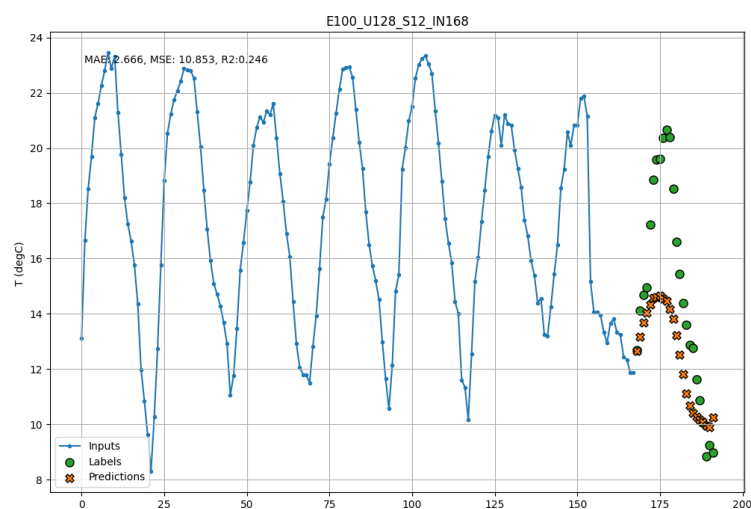
Jak widać na powyższym wykresie model poradził sobie znacznie lepiej i zadowalająco zpredykował dane.

4.3.5 Modele E100 U128 S12 IN168

Jak można zaobserwować na poniższych wykresach, zmiana ilości komórek w warstwie pomimo teoretycznej poprawy rezultatów nie wpłynęła znacznie na zmniejszenie błędu predykcji.

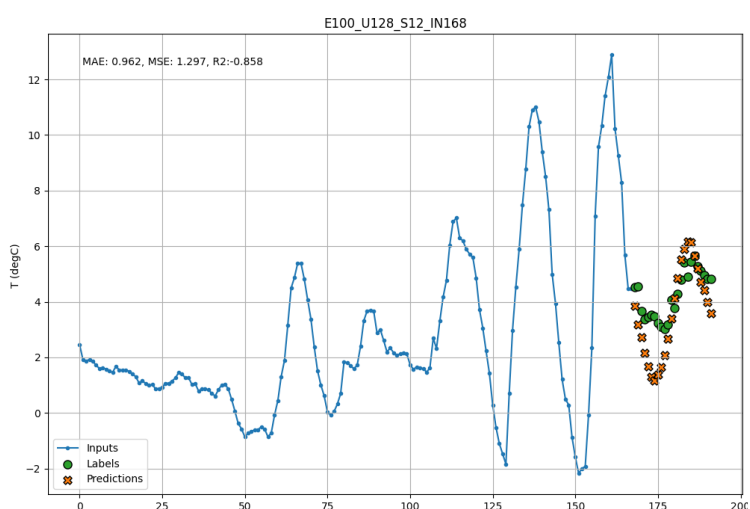


(a) Fragment 1

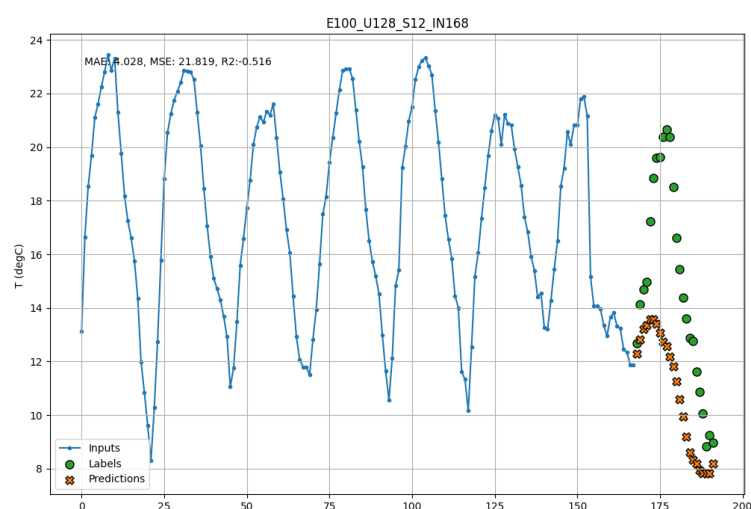


(b) Fragment 2

Rysunek 13: Predykcja GRU



(a) Fragment 1

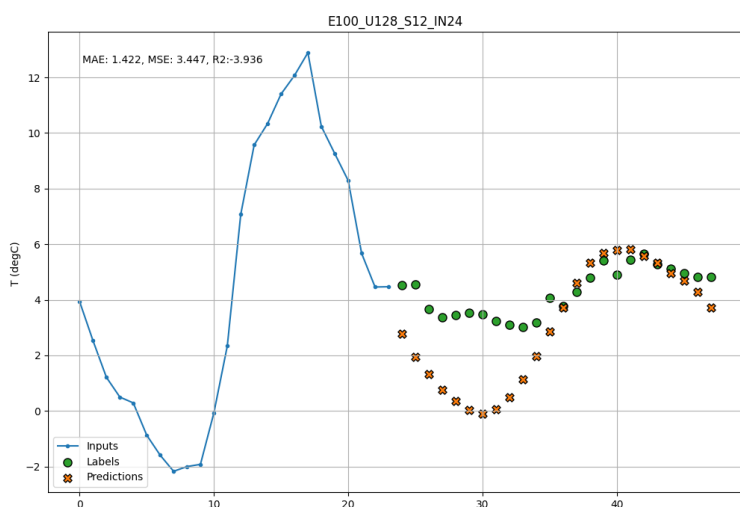


(b) Fragment 2

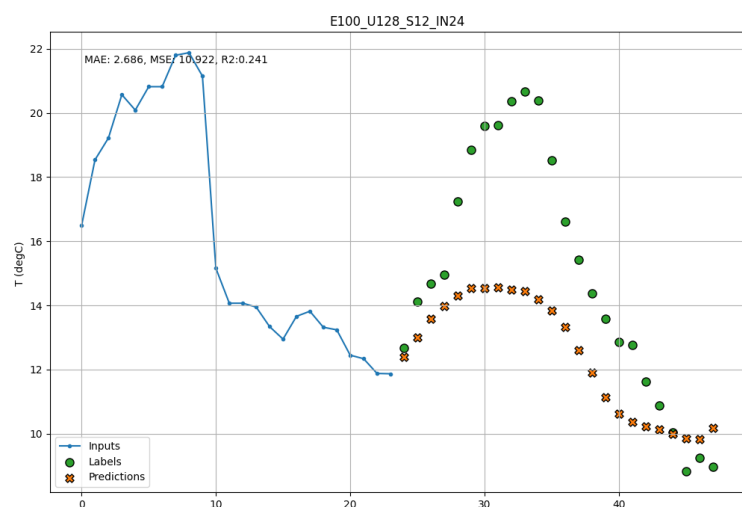
Rysunek 14: Predykcja LSTM

4.3.6 Modele E100 U128 S12 IN124

Model realizujący predykcję i naukę na próbkach z poprzednich 24 godzin bardzo nieznacznie obniżył jakość w porównaniu do modelu biorącego wcześniejsze 7 dni.

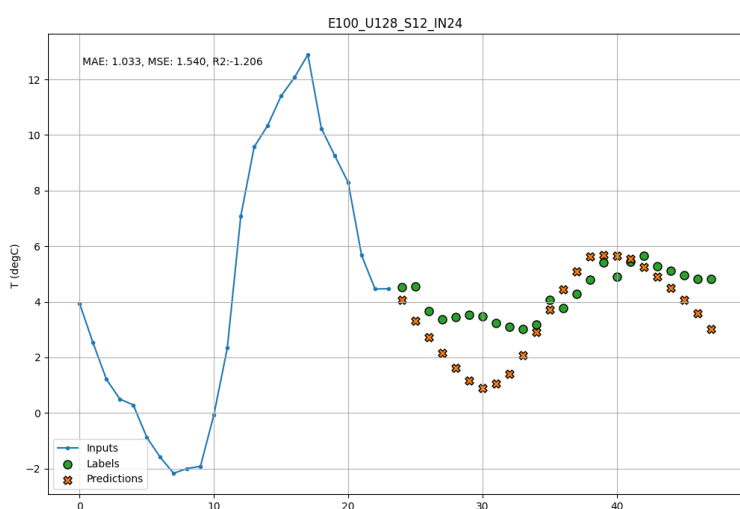


(a) Fragment 1

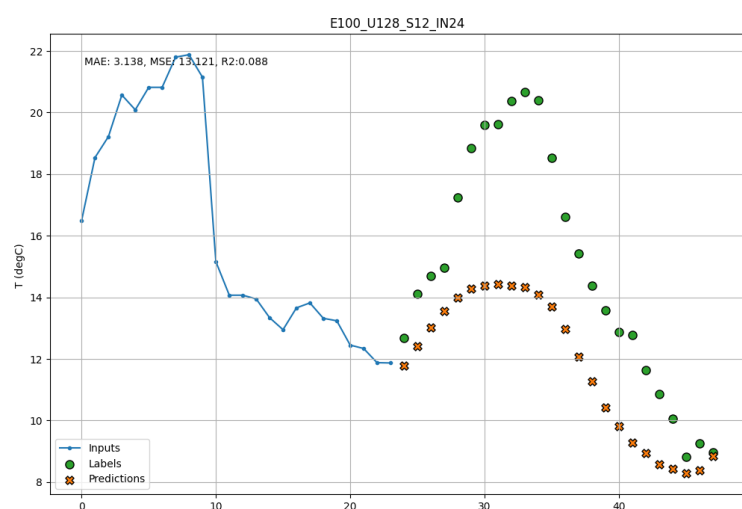


(b) Fragment 2

Rysunek 15: Predykcja GRU



(a) Fragment 1



(b) Fragment 2

Rysunek 16: Predykcja LSTM

5 Wnioski

Badając działanie 24 wyuczonych modeli LSTM oraz takiej samej ilości modeli GRU na zbiorze Jena Climate Dataset otrzymano następujące wnioski:

1. Predykcja pogody okazała się trudnym wyzwaniem dla zaproponowanych architektur sieci. Poprzez swoją nieliniowość bardzo trudno wyuczyć sieć w taki sposób, aby reagowała na nagłe zmiany w temperaturze mające miejsce po stosunkowo równym i okresowym jej przebiegu w ostatnim czasie.
2. W większości przypadku sieć LSTM dawała lepsze wyniki niż GRU, choć zdarzały się wyjątki od tej reguły. Najczęściej sieć LSTM znacznie lepiej poradziła sobie w predykcji fragmentu 1, natomiast miała zbliżone jak nie gorsze wyniki na podstawie błędów MAE, MSE i R2 podczas predykcji fragmentu 2.
3. Najlepsze modele otrzymywały model dla 100 epok uczenia, jednakże nie były to duże różnice w porównaniu z 200 epokami. Zdecydowanie najgorzej poradziły sobie modele z 10 epokami.
4. W przypadku zbioru Danych Jena Climate nie zaobserwowano dużej różnicy w poprawie jakości przy zwiększeniu jednostek sieci z 32 do 128.
5. Zmniejszenie długości sekwencji wejściowej z 168 na 24 nieznacznie pogarsza jakość predykcji. Ponownie w tym przypadku LSTM znacznie lepiej poradził sobie w predykcji fragmentu 1 natomiast GRU trochę lepiej spredykowało fragment 2.
6. Zmianą, która mogłaby poprawić rezultaty byłoby dodanie kolejnych warstw LSTM lub GRU w różnych kombinacjach.